

แผนงานการดำเนินงานการแก้ไขโครงการ ระบบเกมการ์ดเสริมสร้างแนวคิดระบบสารสนเทศ

จัดทำโดย : นายธนภัทร ชูสุวรรณ 6521601857 และนายพีรพุดิ ชีพรับสุข 6521601890

อาจารย์ที่ปรึกษาโครงการ : อาจารย์พิชยภรณ์ พงศกรรังศิลป์

ภายหลังจากการนำเสนอโครงการต่อคณะกรรมการสอบ ผู้จัดทำได้รับข้อเสนอแนะจากอาจารย์ที่ปรึกษาและกรรมการสอบเกี่ยวกับการปรับปรุงระบบฐานข้อมูลและประสบการณ์ผู้ใช้งาน (User Experience) จำนวน 4 ประเด็นหลัก โดยได้รับระยะเวลาในการดำเนินการแก้ไข 1 เดือน (4 สัปดาห์)

1. วัตถุประสงค์ของการแก้ไข

- 1. ยกระดับสถาปัตยกรรมระบบให้เป็น Cloud-based Architecture อย่างสมบูรณ์ โดยให้ฐานข้อมูล Supabase เป็น Single Source of Truth ของระบบ
- 2. เพิ่มระบบ Board State Persistence เพื่อรองรับการกลับเข้าเล่นเกมใหม่เมื่อผู้เล่นขาดการเชื่อมต่อ (Reconnect)
- 3. เปลี่ยนแปลงการจัดเก็บข้อมูลคำถามจาก Local Storage เป็น Online Database เพื่อให้ระบบเป็นออนไลน์ 100%
- 4. พัฒนาโหมดสอนเล่น (Tutorial) ให้ผู้เล่นมีปฏิสัมพันธ์กับ UI จริง และปรับปรุงเว็บไซต์ผู้ดูแลระบบ ให้เป็นมิตรกับผู้ใช้มากขึ้น

สรุปข้อเสนอแนะจากคณะกรรมการสอบ

ลำดับ	ข้อเสนอแนะ	ระดับ ความสำคัญ	ส่วนที่เกี่ยวข้อง
1	ยังไม่มีเก็บสถานะกระดาน (Board State) ลงฐานข้อมูล ส่งผลให้ยังไม่สามารถมีระบบ Reconnect เมื่อผู้เล่นขาดการเชื่อมต่อได้	สูงมาก	Backend / Database
2	ระบบคำถามยังเก็บข้อมูลไว้ในตัวเกม (Local) ไม่ได้เก็บบน Database ออนไลน์ ทำให้ไม่เป็นระบบออนไลน์ 100% — เมื่อเริ่มเกมควร Query คำถามจาก Database แทน	สูงมาก	Backend / Database
3	โหมดสอนเล่น (Tutorial) ยังเป็นแบบอ่านตัวอักษรเท่านั้น ควรปรับให้ผู้เล่นมี Interaction กับ UI ได้จริง	สูง	Frontend / UX
4	หน้าเว็บไซต์สำหรับจัดการคำถาม (Admin Web) ยังไม่เป็นมิตรกับผู้ใช้ ควรมีหน้าจอสําหรับเพิ่มคำถามที่ละข้อ พร้อมกรอกตัวเลือกและเฉลย	สูง	Frontend / Web / Backend / Database

2 . การวิเคราะห์สถานะปัจจุบันของระบบ

โครงสร้างฐานข้อมูลที่มีอยู่

- ปัจจุบันระบบมีตารางจำนวน 6 ตาราง บนฐานข้อมูล Supabase (PostgreSQL) ดังนี้

ตาราง	หน้าที่	สถานะ
player_profiles	เก็บข้อมูลผู้เล่น (ชื่อ, เพชร, MMR, ยศ, ไอเทม)	ใช้งานได้ดี
rooms	เก็บข้อมูลห้องเกม (room_code, host, stat	ใช้งานได้ดี
matchmaking_queue	ระบบจับคู่ผู้เล่นออนไลน์อัตโนมัติ	ใช้งานได้ดี
shop_items	ราคาสินค้าในร้านค้า	ใช้งานได้ดี
daily_quiz_claims	บันทึกการรับรางวัลควิซรายวัน (กัน Claim ซ้ำ)	ใช้งานได้ดี
otp_codes	ระบบ OTP ยืนยันอีเมล (เก็บแบบ SHA-256 Hash)	ใช้งานได้ดี

ฟังก์ชันฝั่ง Server (RPC Functions) ที่มีอยู่

ฟังก์ชัน	หน้าที่	สถานะ
claim_matchmaking_room()	จับคู่ผู้เล่นเข้าห้อง (มี pg_advisory_lock กัน Race Condition)	ใช้งานได้ดี
cancel_matchmaking_search()	ยกเลิกการค้นหาห้อง	ใช้งานได้ดี
get_active_questions()	ดึงคำถามทั้งหมดจาก Patch ที่ Active	ใช้งานได้ดี
import_quiz_patch()	Import คำถามแบบ Bulk จาก JSON	ใช้งานได้ดี

สิ่งที่ยังขาดและจำเป็นต้องเพิ่ม

ระบบที่ขาด	ผลกระทบ
ไม่มีตารางเก็บสถานะกระดานระหว่างเล่น	เมื่อผู้เล่นหลุดการเชื่อมต่อจะไม่สามารถกลับเข้ามาเล่นต่อ (Reconnect) ได้ ต้องออกจากเกมและเสียแมตช์ไป
Unity ยังอ่านคำถามจากไฟล์ Local	ระบบไม่เป็นออนไลน์ 100% เมื่อ Admin เพิ่มหรือแก้คำถามบนฐานข้อมูล ผู้เล่นจะไม่เห็นคำถามใหม่จนกว่าจะอัปเดตตัวเกม
ไม่มี Tutorial แบบ Interactive	ผู้เล่นใหม่ต้องอ่านตัวอักษรเท่านั้น ไม่ได้ลองเล่นจริง
Admin Web ยังไม่มีหน้า Form กรอกคำถาม	Admin ต้องเตรียมไฟล์ JSON แล้วใช้ฟังก์ชัน import เท่านั้น

3.แนวทางการแก้ไขและการออกแบบระบบ

ข้อที่ 1: ระบบเก็บสถานะกระดานและระบบ Reconnect

แนวคิดหลัก (Concept) :

ใช้วิธี "Board State Snapshot" คือการแพ็ค (Serialize) ข้อมูลทุกอย่างบนกระดาน (เหรียญทรัพยากร, การ์ดบนกระดาน, คะแนนผู้เล่น, ลำดับไพ่ในกอง ฯลฯ) ให้อยู่ในรูปแบบ JSON ก้อนเดียว แล้วบันทึกลงฐานข้อมูลทุกครั้งที่มีการเปลี่ยนแปลงสำคัญ (เช่น จบเทิร์นของผู้เล่นแต่ละคน)

กระบวนการทำงาน (Flow) :

กระบวนการบันทึก (Save):

- เมื่อเกมเริ่ม (Setup Phase) → Master Client สร้างแถว (Row) ใหม่ในตาราง `match_sessions` โดยผูก `room_code` จากตาราง `rooms` ที่มีอยู่แล้ว
- เมื่อผู้เล่นจบเทิร์น → Master Client เรียกฟังก์ชัน `Serialize()` เพื่อแปลงข้อมูลบนกระดานทั้งหมดเป็น JSON → ส่งคำสั่ง UPDATE ไปยัง `match_sessions.board_state`
- เมื่อเกมจบ → เปลี่ยนสถานะ `match_sessions.status` เป็น 'finished'

กระบวนการ Reconnect:

1. เมื่อผู้เล่นเปิดเกมและ Login สำเร็จ → ระบบ Query ตาราง match_sessions เพื่อเช็คว่ามี Active Session ที่ตนเองเป็นสมาชิกอยู่หรือไม่
2. หากพบ Active Session → แสดงปุ่ม "กลับเข้าเกม" บนหน้า Lobby
3. เมื่อกดปุ่ม → ระบบดึง board_state (JSON) จาก Database → เรียกฟังก์ชัน Deserialize() เพื่อสร้างกระดานจากข้อมูลที่บันทึกไว้ → เชื่อมต่อเข้า Photon Room เดิม

กระบวนการ เมื่อผู้เล่นหลุด (Disconnect & Reconnect แบบไม่หยุดเกม):

1. เมื่อ Photon Fusion ตรวจสอบว่าผู้เล่นคนใดคนหนึ่งหลุดออกจากห้อง เกมจะไม่หยุด ผู้เล่นที่เหลือยังเล่นต่อได้ตามปกติ โดยระบบแสดงเพียงสถานะ “ผู้เล่น X กำลังเชื่อมต่อใหม่...” บนหน้าจอ ซึ่งไม่บล็อกการเล่น
2. ในแต่ละเทิร์นมีตัวจับเวลา (Turn Timer) กำกับอยู่แล้ว หากถึงเทิร์นของผู้เล่นที่หลุดและหมดเวลา ระบบจะเล่นแทนแบบปลอดภัยให้อัตโนมัติ เช่น ข้ามเทิร์น แล้วส่งต่อให้ผู้เล่นคนถัดไป ทำให้เกมดำเนินต่อไปโดยไม่ติดขัด
3. ระหว่างนั้นผู้เล่นที่หลุดสามารถกลับเข้าห้องเดิมได้ทุกเมื่อด้วย Quick Rejoin (ReconnectAndRejoin) ของ Photon โดยตั้งค่า PlayerTTL ให้นานพอ (เช่น 2-3 นาที) และใช้ ConnectionToken เพื่อรับสิทธิ์ควบคุมผู้เล่นเดิมกลับคืน โดยไม่ถูกสร้างเป็นผู้เล่นใหม่
4. เมื่อกลับเข้ามา ระบบจะดึง board_state ก่อนล่าสุดจากตาราง match_sessions มา Deserialize เพื่อให้ผู้เล่นเห็นสถานะปัจจุบันของเกมทันที (ไม่ใช่จุดที่หลุดไป) จึงเล่นต่อได้เลยโดยไม่ต้องให้ผู้เล่นคนอื่นหยุดรอ
5. หากผู้เล่นถูกเล่นแทนติดต่อกันหลายเทิร์น (เช่น 2-3 เทิร์น) หรือยาวนานเกินกำหนด (เช่น 3 นาที) ระบบจึงจะปรับเป็น Forfeit (ยอมแพ้) แล้วหัก MMR ตามปกติ พร้อมอัปเดต match_sessions.status เป็น ‘abandoned’

เหตุผลในการเลือกวิธี Snapshot แบบ JSONB

- ความยืดหยุ่น: สถานะกระดานมีโครงสร้างไม่คงที่ (จำนวนผู้เล่น 2-4 คน, จำนวนการ์ดในมือไม่เท่ากัน) การแตกตารางหลายตารางจะซับซ้อนเกินไป
- ประสิทธิภาพ: PostgreSQL มี JSONB ที่ Query และ Index ได้เร็ว
- ความง่ายในการพัฒนา: Serialize()/Deserialize() สะดวกทั้งฝั่ง Unity (C#) และฝั่ง Database

ข้อที่ 2: เปลี่ยนระบบคำถามเป็น Online 100%

สถานะปัจจุบัน :

ตัวเกม Unity ยังอ่านข้อความจากไฟล์ JSON ที่ฝังมากับตัว Build (Local Asset)

แนวทางแก้ไข :

- แก้โค้ด Unity: ในขั้นตอน Setup Phase (ก่อนเริ่มเกม) ให้ Master Client เรียกฟังก์ชัน get_active_questions() ผ่าน Supabase SDK เพื่อดึงข้อมูลคำถามทั้งหมดจาก Database
- แปลงข้อมูล: แปลง JSON Response ที่ได้เป็น List<QuizQuestion> ใน C# แล้วเก็บไว้ในหน่วยความจำ (RAM)
- ใช้งานลอจิกเดิม: ฟังก์ชันสุ่มคำถามระหว่างเกม ยังคงสุ่มจาก List ใน RAM ได้เหมือนเดิม — แก้ไขเฉพาะจุดโหลดข้อมูลเท่านั้น ลอจิกอื่นไม่ต้องเปลี่ยน

ผลลัพธ์ที่คาดหวัง :

- Admin เพิ่มคำถามใหม่ผ่านเว็บ → ผู้เล่นเปิดเกมครั้งถัดไป → เห็น คำถามใหม่ทันที โดยไม่ต้องอัปเดตตัวเกม
- ระบบเป็น Online 100% ตามข้อเสนอแนะของอาจารย์

ข้อที่ 3: โหมดสอนเล่นแบบ Interactive (Tutorial Mode)

แนวคิดหลัก :

เปลี่ยนจากการ "อ่านข้อความ" เป็น "เรียนรู้โดยการลงมือทำ" (Learn by Doing) โดยใช้เทคนิค UI Masking ที่ทำให้หน้าจอมืดลง และสว่างเฉพาะปุ่มที่ผู้เล่นต้องกด

รายละเอียดขั้นตอนการสอน (Tutorial Steps)

ลำดับ	การสอน	UI ที่เปิดให้กด	ข้อความอธิบาย
1	สอนตอบคำถาม	เฉพาะปุ่มตัวเลือก วิชา	- "เมื่อเริ่มเกม จะมีคำถามขึ้นมาให้ตอบเพื่อแย่งลำดับเทิร์นกัน" - "โดยคนที่ตอบได้ไวสุดและถูกต้องจะได้เล่นคนแรกกับได้เหรียญพิเศษ" - "ส่วนคนที่ตอบถูกช้าลงมากก็จะได้เทิร์นถัดไปตามลำดับการตอบ " - "คนที่ตอบผิดจะอยู่ถัดมาจากคนที่ตอบถูก" - "ลองกดตอบข้อนี้ดูสิ" - "คุณได้เริ่มคนแรกและได้เหรียญพิเศษด้วย"
2	สอนหยิบเหรียญทรัพยากร อย่างละ 1 เหรียญ	เฉพาะปุ่มเหรียญ ทรัพยากรกองกลาง	- "เหรียญทรัพยากรจำเป็นสำหรับการซื้อการ์ด โดยเหรียญที่หยิบได้จะมี 5 แบบ และแบบพิเศษ 1 แบบ และสามารถหยิบได้พร้อมกัน 3 เหรียญ" - "ลองหยิบเหรียญทรัพยากรอย่างละเหรียญจากกองกลาง"

3	สอนกดยกเลิกหีบเหรียญ	เฉพาะปุ่มยกเลิกหีบเหรียญ	"หากกดหีบเหรียญผิดอัน สามารถกดยกเลิกได้ที่ปุ่มนี้"
4	สอนหีบเหรียญเหรียญแบบเดียวกัน 2 เหรียญ	เฉพาะปุ่มเหรียญ RAM	- "และถ้าเหรียญแต่ละแบบ มีเท่ากับหรือมากกว่า 4 เหรียญ จะสามารถหีบเหรียญแบบเดียวกันได้ 2 เหรียญ แต่จะหีบได้แค่ 2 เหรียญเท่านั้น" - "ลองหีบเหรียญ RAM 2 เหรียญสิ"
5	สอนกดจบเทิร์น	เฉพาะปุ่มจบเทิร์น	"กดจบเทิร์นเพื่อส่งต่อให้ผู้เล่นคนถัดไป"
6	สอนซื้อการ์ด	เฉพาะการ์ดที่ล็อกไว้ให้ซื้อ	- "การซื้อการ์ดต้องกด 1 ครั้งที่การ์ดที่ต้องการซื้อ" - "ลองซื้อการ์ดใบนี้ดู ใช้ทรัพยากรจากที่เก็บมา" - "หลังจากที่ซื้อการ์ดแล้ว การ์ดแต่ละสีที่ซื้อมาจะสามารถลดการใช้ทรัพยากรของสีนั้นๆ ในการซื้อการ์ดใบอื่นได้"
7	สอนจองการ์ด	เฉพาะการ์ดที่ล็อกไว้ให้จอง	- "การจองการ์ดต้องกดค้างที่การ์ดที่ต้องการซื้อ" - "ลองจองการ์ดใบนี้ดู" - "การ์ดที่จองจะมาอยู่ตรงนี้ และได้เหรียญพิเศษมา 1 เหรียญ โดยเหรียญพิเศษจะสามารถใช้แทนเหรียญอะไรก็ได้ในการซื้อการ์ด"
8	ลองให้รวบรวมเหรียญทรัพยากรอีกรอบ	เฉพาะปุ่มเหรียญทรัพยากรกองกลาง	- "ลองรวบรวมเหรียญที่ขาดเพื่อมาซื้อการ์ดที่จองไว้"
9	สอนซื้อการ์ดที่จอง	เฉพาะการ์ดที่จองไว้	- "ทรัพยากรครบแล้ว ลองกดที่การ์ดที่จองไว้" - "สามารถซื้อการ์ดที่จองได้ที่ตรงนี้"
10	วิธีชนะเกม	-	- "คุณสามารถชนะเกมนี้ได้โดยที่รวบรวมดาวให้ครบ 20 ดวงก่อนผู้เล่นคนอื่น โดยดาวจะได้มา 2 วิธี" - "วิธีที่ 1 จากการซื้อการ์ดโดยการกดแต่ละใบจะมีดาวบอกอยู่ด้านขวาบนของการ์ด" - "วิธีที่ 2 จะได้จากการซื้อการ์ดแต่ละสีตามที่กำหนดได้ครบก่อนผู้เล่นคนอื่นจะได้ดาวตามที่กำหนดไว้ไป" - "ถ้าเข้าใจแล้วไปลองเล่นเกมจริงกันเลย หรือสามารถไปกดอ่านวิธีการเล่นได้ตรงหน้าเมนู"

การพัฒนาทางเทคนิค

- สร้าง Unity Scene ใหม่ชื่อ TutorialScene (เล่นแบบ Offline ไม่ต้องเชื่อมต่อ Photon)
- สร้าง Script TutorialManager.cs ใช้ State Machine ในการควบคุมลำดับขั้นตอน
- ใช้ Canvas UI วาง Panel สีดำ (Alpha 75%) ทั้ทั้งจอ เฉพาะตำแหน่งปุ่มเป้าหมายด้วย RectMask2D หรือ Custom Shader
- ล็อก Input ปุ่มอื่นทั้งหมดด้วย button.interactable = false ยกเว้นปุ่มเป้าหมาย

ข้อที่ 4: ปรับปรุงเว็บไซต์ผู้ดูแลระบบ (Admin Web)

สถานะปัจจุบัน

มีฟังก์ชัน import_quiz_patch() ที่รองรับการ Import คำถามแบบ Bulk (JSON Array) แล้ว แต่ยังขาดหน้าจอ UI ให้ Admin สามารถเพิ่มคำถามทีละข้อผ่าน Form ได้สะดวก

หน้าจอที่จะพัฒนาเพิ่ม

หน้าที่ 1: แบบฟอร์มเพิ่มคำถามทีละข้อ (Manual Form)

ฟิลด์	ประเภท UI	รายละเอียด
โจทย์คำถาม	Textarea	กรอกข้อความคำถาม
ตัวเลือก ก.	Text Input	กรอกตัวเลือกที่ 1
ตัวเลือก ข.	Text Input	กรอกตัวเลือกที่ 2
ตัวเลือก ค.	Text Input	กรอกตัวเลือกที่ 3
ตัวเลือก ง.	Text Input	กรอกตัวเลือกที่ 4
เฉลย	Dropdown	เลือก ก. / ข. / ค. / ง.
หมวดหมู่	Dropdown	ทั่วไป, เครือข่าย, ฐานข้อมูล ฯลฯ
ระดับความยาก	Dropdown	ง่าย / ปานกลาง / ยาก
ปุ่ม "บันทึก"	Button	INSERT ลงตาราง quiz_questions

หน้าที 2: ตารางแสดงคำถามทั้งหมด (Data Table)

- แสดงรายการคำถามจากฐานข้อมูลพร้อมตัวกรอง (Filter) ตามหมวดหมู่และระดับ
- มีปุ่ม "แก้ไข" (UPDATE) และ "ลบ" (DELETE) ในแต่ละแถว
- รองรับการค้นหาคำถามด้วย Keyword

หน้าที่ 3: Import คำถามแบบ Bulk (มีอยู่แล้ว — ปรับปรุง UI)

- ปรับปรุงหน้าเดิมให้มีตัวอย่าง JSON Format แสดงไว้ให้ Admin ดู
- แสดง Preview ก่อน Import จริง

4.การออกแบบฐานข้อมูลที่ปรับปรุง

ภาพรวมการเปลี่ยนแปลง

ในการแก้ไขครั้งนี้ โครงสร้างฐานข้อมูลเดิมทั้ง 8 ตารางยังคงอยู่ครบถ้วนโดยไม่มีการแก้ไขหรือลบ เนื่องจากระบบที่มีอยู่ถูกออกแบบมาอย่างเหมาะสมแล้ว สิ่งที่ดำเนินการเพิ่มเติมมีเพียง การเพิ่มตารางใหม่ 1 ตาราง เพื่อรองรับฟีเจอร์ที่ขาดหายไป คือระบบเก็บสถานะกระดานและ Reconnect

ตารางฐานข้อมูลทั้งหมด (ก่อนและหลังแก้ไข)

ลำดับ	ชื่อตาราง	สถานะ	บทบาทในระบบ
1	player_profiles	คงเดิม	จัดเก็บข้อมูลโปรไฟล์ผู้เล่นทั้งหมด ได้แก่ ชื่อผู้ใช้ ค่า MMR จำนวนเพชร สถิติชนะ/แพ้ ไอเทมที่ครอบครอง และกรอบที่สวมใส่อยู่
2	rooms	คงเดิม	จัดเก็บข้อมูลห้องเกมที่ระบบสร้างขึ้น ประกอบด้วยรหัสห้อง ชื่อ Host จำนวนผู้เล่น และสถานะห้อง
3	matchmaking_queue	คงเดิม	จัดการคิวรอจับคู่ผู้เล่น เพื่อป้องกันปัญหาการเข้าห้องพร้อมกันและรองรับการยกเลิกการค้นหา
4	shop_items	คงเดิม	เก็บรายการสินค้าและราคาในร้านค้า เพื่อป้องกันการโกงจากฝั่งผู้เล่น
5	daily_quiz_claims	คงเดิม	บันทึกประวัติการรับรางวัลควิซรายวัน เพื่อป้องกันผู้เล่นรับรางวัลซ้ำในวันเดียวกัน

6	quiz_patches	คงเดิม	จัดเก็บข้อมูลชุดคำถาม รองรับการเปิด-ปิดชุดคำถามได้ทันที
7	otp_codes	คงเดิม	จัดเก็บรหัสความปลอดภัย OTP เพื่อยืนยันตัวตนผ่านอีเมล
8	quiz_questions	ปรับปรุงใหม่	ปรับปรุงระบบคลังคำถามควิซทั้งหมด มีการจัดหมวดหมู่ ระดับความยาก และเก็บตัวเลือก และทำงานแบบ ออนไลน์จริง 100%
9	match_sessions	เพิ่มใหม่	เก็บสำเนาสถานะกระดานเกมทุกครั้งที่จบเทิร์น เพื่อรองรับระบบ Reconnect

รายละเอียดตารางใหม่: match_sessions

บทบาทและความสำคัญ

ตาราง match_sessions ถูกออกแบบมาเพื่อทำหน้าที่เป็น "สมุดบันทึกสถานะเกม" ที่อัปเดตตัวเองทุกครั้งที่มีการเปลี่ยนแปลงสำคัญบนกระดาน ทำให้เมื่อผู้เล่นขาดการเชื่อมต่อและกลับเข้ามา ระบบสามารถดึงข้อมูลจากตารางนี้เพื่อฟื้นฟูกระดานให้กลับไปยังจุดที่เล่นค้างไว้ได้อย่างถูกต้อง

ชื่อคอลัมน์	ชนิดข้อมูล	คำอธิบาย
id	รหัสเฉพาะ (UUID)	รหัสประจำ Session สร้างโดยอัตโนมัติ
room_code	ข้อความ (Text)	รหัสห้องเกม เชื่อมโยงกับตาราง rooms เพื่อไม่ให้ข้อมูลซ้ำซ้อน
board_state	JSON แบบประหยัค (JSONB)	สถานะกระดานเกมทั้งหมด บันทึกในรูปแบบ JSON (ดูรายละเอียดในข้อ 5.4)
status	ข้อความ (Text)	สถานะของแมตช์ มี 3 ค่า ได้แก่ กำลังเล่น (active), จบปกติ (finished), ผู้เล่นหลุดถาวร (abandoned)
created_at	เวลา (Timestamp)	วันเวลาที่เริ่ม Session
updated_at	เวลา (Timestamp)	วันเวลาที่อัปเดตล่าสุด อัปเดตอัตโนมัติผ่านกลไกของฐานข้อมูล

การออกแบบด้านความปลอดภัย ตาราง match_sessions ถูกกำหนดสิทธิ์การเข้าถึงอย่างรัดกุมดังนี้:

- ผู้เล่นทั่วไป: มีสิทธิ์อ่านได้เฉพาะ Session ที่มีสถานะกำลังเล่น (active) เท่านั้น เพื่อใช้สำหรับตรวจสอบการ Reconnect เท่านั้น ไม่มีสิทธิ์แก้ไขข้อมูลเพื่อป้องกันการโกง
- เซิร์ฟเวอร์ส่วนกลาง: มีสิทธิ์เขียน อ่าน แก้ไข และลบข้อมูลได้ทั้งหมด เพื่อทำหน้าที่บันทึกสถานะกระดานได้อย่างสมบูรณ์

เหตุผลที่เลือกใช้รูปแบบ JSON ในการจัดเก็บสถานะกระดาน

แทนที่จะสร้างตารางแยกย่อยจำนวนมากเพื่อเก็บตำแหน่งการ์ดแต่ละใบหรือเหรียญแต่ละสี ทางผู้พัฒนาได้เลือกใช้ประเภทข้อมูล JSON ในการจัดเก็บด้วยเหตุผลดังนี้:

1. โครงสร้างข้อมูลที่ยืดหยุ่น: สถานะกระดานมีการเปลี่ยนแปลงตลอดเวลา จำนวนการ์ดในมือผู้เล่นไม่เท่ากัน การเก็บรวมเป็น JSON Object ช่วยลดความซับซ้อนของโครงสร้างฐานข้อมูล
2. ประสิทธิภาพสูง: ฐานข้อมูลรองรับข้อมูลแบบ JSON ซึ่งมีการจัดทำดัชนี (Indexing) ทำให้สามารถสืบค้นข้อมูลได้อย่างรวดเร็ว
3. ลดความซับซ้อนในการพัฒนา: โค้ดฝั่งผู้เล่นสามารถทำงานร่วมกับ JSON ได้โดยตรงไปตรงมา ผ่านกระบวนการแปลงข้อมูล โดยไม่ต้องเขียนคำสั่งเชื่อมโยงหลายตาราง

โครงสร้างข้อมูลที่จัดเก็บในกระดาน

ข้อมูลที่จะถูกบรรจุและบันทึกในรูปแบบ JSON ประกอบด้วยองค์ประกอบหลักที่จำเป็นต่อการฟื้นฟูกระดานทั้งหมด ได้แก่:

หมวดหมู่ข้อมูล	รายละเอียดที่จัดเก็บ	วัตถุประสงค์ในการฟื้นฟู
สถานะหลักของเกม	รอบที่กำลังเล่น, ดัชนีผู้เล่นที่กำลังเดิน, และลำดับการเดิน	เพื่อให้เกมรู้ว่าถึงเทิร์นของใครเมื่อผู้เล่นกลับเข้ามา
ทรัพยากรกองกลาง	จำนวนเหรียญในกองกลางครบทุกสี	เพื่อจัดวางเหรียญกองกลางให้มีจำนวนเท่าเดิม
สถานะกองการ์ด	ลำดับการ์ดในกองแต่ละระดับ และตัวชี้ตำแหน่ง (Deck Pointers)	เพื่อรักษาลำดับไพ่ในกอง ไม่ให้ถูกสับใหม่หลัง Reconnect
สถานะบนกระดาน	รหัสการ์ด (Card ID) ที่วางหงายหน้าอยู่บนกระดานแต่ละช่อง และการ์ดขุนนางที่เหลืออยู่	เพื่อแสดงผลการดบนกระดานให้ถูกต้องตรงตำแหน่งเดิม
สถานะผู้เล่นทุกคน	คะแนนรวม, เหรียญทรัพยากรในมือ, โบนัสทรัพยากรถาวร, การ์ดที่ครอบครอง, การ์ดที่จองไว้, และสถานะการออนไลน์	เพื่อจัดเตรียมมือของผู้เล่นทุกคน รวมทั้งผู้เล่นที่ไม่ได้หลุด ให้ตรงตามจริง
ข้อมูลคำถาม	รายการรหัสคำถามที่ถูกใช้งานไปแล้วในแมตช์นั้น	เพื่อป้องกันการสุ่มได้คำถามซ้ำหลังจากการ Reconnect

5. แผนการดำเนินงานรายสัปดาห์

สัปดาห์ที่ 1: ระบบคำถามออนไลน์ 100% และเว็บผู้ดูแลระบบ (ข้อ 2 + ข้อ 4)

วันที่	งานที่ต้องทำ	ผลลัพธ์ที่คาดหวัง
วันที่ 1-2	แก้ไขโค้ด Unity (QuizManager.cs) ให้เรียก get_active_questions() ผ่าน Supabase SDK แทนการอ่านไฟล์ JSON	เกมโหลดคำถามจาก DB ได้สำเร็จ
วันที่ 3-4	พัฒนาหน้า Admin Web: แบบฟอร์มเพิ่มคำถามที่ละข้อ (Create Form)	Admin กรอกคำถาม + ข้อยส์ + เฉลย แล้วกด "บันทึก" ได้
วันที่ 5-6	พัฒนาหน้า Admin Web: ตารางแสดง/แก้ไข/ลบคำถาม (Data Table + Edit/Delete)	Admin กรอกคำถาม + ข้อยส์ + เฉลย แล้วกด "บันทึก" ได้
วันที่ 7	ทดสอบ End-to-End: เพิ่มคำถามจากเว็บ → เปิดเกม → เจอคำถามใหม่	ระบบ Online 100% ทำงานสมบูรณ์

สัปดาห์ที่ 2: ระบบบันทึกสถานะกระดาน (ข้อ 1 — ส่วน Serialize)

วันที่	งานที่ต้องทำ	ผลลัพธ์ที่คาดหวัง
วันที่ 1	รัน Migration สร้างตาราง match_sessions บน Supabase	ตารางพร้อมใช้งานพร้อม RLS
วันที่ 2-3	เขียน BoardStateSerializer.cs ฟังก์ชัน Serialize: กวาดข้อมูลกระดาน → แพ็คเป็น JSON	ได้ JSON สมบูรณ์จากข้อมูลจริง
วันที่ 4-5	เขียนลอจิก INSERT ตอนเริ่มเกม + UPDATE ตอนจบเทิร์น	ข้อมูลกระดานถูกบันทึกลง DB ทุกเทิร์น
วันที่ 6-7	ทดสอบ: เล่นเกม 5 เทิร์น → ตรวจสอบข้อมูลใน Supabase Dashboard	JSON ใน DB ตรงกับสถานะกระดานจริง

สัปดาห์ที่ 3: ระบบ Reconnect (ข้อ 1 — ส่วน Deserialize + Disconnect)

วันที่	งานที่ต้องทำ	ผลลัพธ์ที่คาดหวัง
วันที่ 1-2	เขียน BoardStateSerializer.cs ฟังก์ชัน Deserialize: อ่าน JSON → สร้างกระดานใหม่	กระดานถูกสร้างขึ้นจาก JSON ได้ถูกต้อง
วันที่ 3	เพิ่มลอจิกตอน Login: เช็ค Active Session + แสดงปุ่ม "กลับเข้าเกม"	ผู้เล่นเห็นปุ่ม Reconnect เมื่อมี Session ค้าง
วันที่ 4-5	เขียน DisconnectManager.cs: ตรวจสอบ Disconnect + เล่นแทนอัตโนมัติเมื่อหมดเวลาเทิร์น + Quick Rejoin + Forfeit Logic	เกมเดินต่อได้แม้มีคนหลุด และผู้เล่นกลับเข้ามาเล่นต่อได้
วันที่ 6-7	ทดสอบ Reconnect: ปิด Wi-Fi → เปิดใหม่ภายใน 60 วิ → เกมกลับมาที่จุดเดิม	ระบบ Reconnect ทำงานสมบูรณ์

สัปดาห์ที่ 4: โหมดสอนเล่น Interactive + ทดสอบรวม (ข้อ 3)

วันที่	งานที่ต้องทำ	ผลลัพธ์ที่คาดหวัง
วันที่ 1-2	สร้าง TutorialScene + TutorialManager.cs (State Machine)	Tutorial เล่นได้มี UI รองรับ
วันที่ 3-4	สร้าง UI Masking (Panel เงาระบุ + ข้อความอธิบาย) ทั้ง 6 Steps	ผู้เล่นกดปุ่มตามลำดับจนจบ Tutorial
วันที่ 5	ทดสอบ Tutorial จากมุมมองผู้เล่นใหม่ที่ไม่เคยเล่นเกมมาก่อน	ผู้เล่นใหม่เข้าใจวิธีเล่นได้
วันที่ 6-7	ทดสอบระบบทั้งหมดรวมกัน (Integration Test) ตาม Test Plan ในหัวข้อ 7	ทุกระบบทำงานร่วมกันได้

6.แผนการทดสอบระบบ

- การทดสอบนี้มุ่งเน้นไปที่การรับรองการทำงานของระบบใหม่ที่ถูกเพิ่มเข้ามา เพื่อสร้างความมั่นใจก่อนการประเมินผล

ลำดับ	กรณีทดสอบ	ขั้นตอน	ผลลัพธ์ที่คาดหวัง
1	เพิ่มคำถามจากเว็บ Admin แล้วเกมดึงได้	เพิ่มคำถามจาก Form → เปิดเกม → เริ่มเกมใหม่	เจอคำถามใหม่ในเกมทันที
2	ลบคำถามจากเว็บ Admin แล้วเกมไม่แสดง	ลบคำถามจาก Data Table → เปิดเกมใหม่	คำถามที่ลบไม่ปรากฏในเกม
3	สถานะกระดานถูกบันทึกทุกเทิร์น	เล่นเกม 5 เทิร์น → ตรวจสอบ DB	JSON ใน DB ตรงกับสถานะจริง
4	Reconnect สำเร็จ กลับเข้าเล่นต่อได้	ปิด Wi-Fi → รอ 30 วิ → เปิด Wi-Fi → กดกลับเข้าเกม	กลับเข้าเกมได้ เห็นสถานะปัจจุบันของเกม โดยผู้เล่นคนอื่นไม่ต้องหยุดรอ
5	Reconnect หายนานเกินเกณฑ์ → Forfeit	ปิด Wi-Fi → ทิ้งคั้งไว้จนถูกเล่นแทนหลายเทิร์นหรือหายเกิน 3 นาที	ผู้เล่นที่หลุดถูกปรับแพ้, MMR หัก
6	Tutorial เล่นจนจบได้	เข้าโหมด Tutorial → กดตามสเต็ป 1-6	ผ่าน Tutorial ครบทุก Step
7	Tutorial ล็อก Input ปุ่มที่ไม่ใช่เป้าหมาย	กดปุ่มอื่นที่ไม่ใช่เป้าหมายในแต่ละ Step	ปุ่มไม่ตอบสนอง